

VoiceQuote

口述一句中文询盘，出一份英文报价单

A voice-first quotation agent for Chinese export sales. The salesperson speaks the buyer's inquiry in Chinese — the agent transcribes it, calls a deterministic pricing engine, and hands back a client-ready English quotation.

AssemblyAI Voice Agent API

JSON-Schema tool calling

Verified end-to-end on a live deployment

THE PROBLEM

A 30-second inquiry costs 40 minutes of desk work

A Chinese factory salesperson takes the inquiry by phone or voice message, then retypes it, looks up specs, computes volumetric freight and US duty, and writes the quote in English. The three mistakes that actually lose money are all mechanical.

1 cbm $\approx$ 167 kg

Volumetric weight

Phone cases are light-bulky cargo.
Quoting on actual weight under-prices every shipment.

17.6%

US duty + Section 301

Duty sits on CIF value; Section 301 stacks on top and changes without notice.

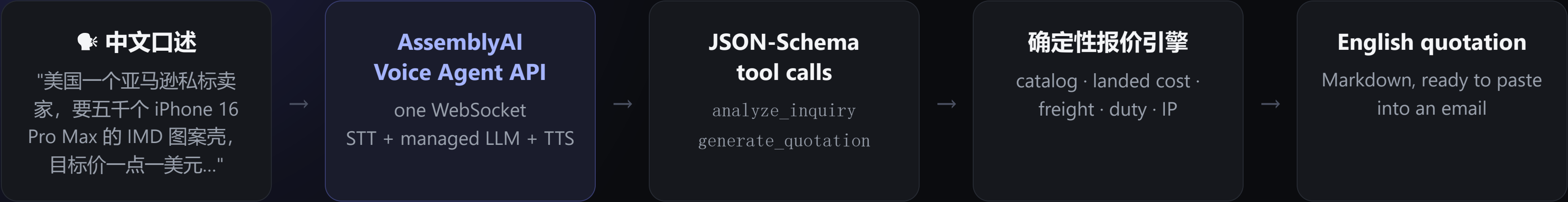
IP chain

Printed goods

Artwork needs a written authorisation chain. Quoting without it is the category's #1 cause of loss.

THE SOLUTION

Speak once. Get a quote you can send.



Voice in, voice out

The agent speaks a 2–3 sentence summary back in Chinese and says the quote is on screen. It is explicitly forbidden from reading tables aloud.

Numbers never come from the model

Every price, MOQ, freight figure and duty rate comes from the deterministic engine. The system prompt forbids the model from inventing any number.

Everything we use from AssemblyAI

CAPABILITY	HOW VOICEQUOTE USES IT	OBSERVED
Realtime STT (Universal)	Chinese speech, mixed CN/EN terms — biased with <code>keyterms</code> + a scene <code>transcription_prompt</code>	6 segments / 14.2s audio
Managed LLM routing	Decides which tool to call and what to say; no external LLM dependency	no third-party TTFB
Turn detection / VAD	<code>min_silence</code> 1800 so one spoken paragraph lands as a single turn	6 segments → 1 turn
Barge-in	<code>interrupt_response: true, interruption_delay: 200</code>	live-configurable
TTS	alba, 24 kHz PCM16 streamed back over the same socket	1,524 chunks
JSON-Schema tool calling	Two tools executed client-side against our own <code>/api/quote*</code>	2 calls / 2 results
Session artifacts	Official <code>timeline</code> + <code>audio</code> used as the evidence for this deck	4 turns

HARD PROBLEM #1

The tool result was never sent — and it looked like a server fault

Our agent kept answering "不好意思，系统刚才卡了一下" after ~32 seconds. The symptom pointed at AssemblyAI; the cause was a local timing rule we got wrong.

What we assumed

A tool-call turn's `reply.done` would carry the `fc-<call_id>` we saw in `tool.call` — so we gated `tool.result` on that prefix.

What the timeline showed

The tool turn's `turn_id` is `resp_...`, not `fc-...`. The gate never opened, so `tool.result` sat in a queue while the tool timed out at 30 s.

```
// 官方 timeline 里那个工具轮的 turn_id — 根本不是 fc- 前缀
turn_id: 'resp_4593193aeda54b1f9c7822d01431f8c4'

// 正确做法: 用 reply.done 的**序号**判断, 而不是 reply_id 的前缀
const seqAtCall = replyDoneSeq;
if (replyDoneSeq > seqAtCall) sendResult(r); // 该轮已结束 → 立刻回传
else readyResults.push(r); // 仍在飞 → 排队, 等下一个 reply.done 冲掉
```

Rule send `tool.result` when `reply.done` is the latest event received

HARD PROBLEM #2

The customer said 5,000. The quote said 1,000.

This is a Chinese-speech product, and AssemblyAI correctly transcribes Chinese numerals — "五千个", "一点一美元". Our deterministic parser only matched Arabic digits, so both values silently fell through to defaults. In a quotation product, a wrong number is the whole failure.

1,000 pcs

Before

五千个 → no match → default 1000.

一点一美元 → no match → target price `null`, so the "target is unreachable" warning never fired.

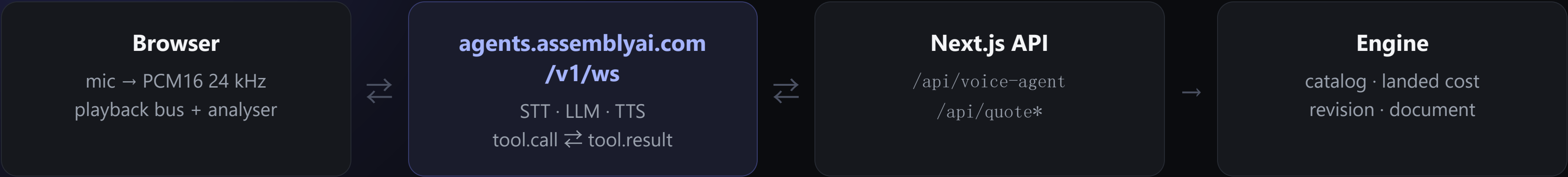
5,000 pcs

After

`parseCnNumber()` handles 十/百/千/万/亿, colloquial elision (一千五 = 1500) and decimals (一点一 = 1.1). Target price parses, and the warning fires correctly.

```
// 数量候选必须"全部收集 + 过滤 + 取最大", 不能只取第一个匹配:  
// "美国一个亚马逊私标卖家, 要五千个..." 里第一个被命中的是"一个"  
looksLikeQuantity(cn) => cn.length >= 2 || /[十百千万亿]/.test(cn) // "一" 被挡掉
```

Thin browser, deterministic core



Keys never reach the browser

The API key stays server-side. The browser receives a single-use, short-lived token (300 s exchange window, 1200 s session cap).

Tools run client-side

The pricing engine lives in the same app, so tools call our own API routes — no public round-trip and no 8 KiB response ceiling.

Billing discipline

Explicit `session.end` stops billing immediately; a bare socket close keeps billing alive for a 30 s resume window.

MEASURED RESULTS

Live production run, real speech, no mocks

14.22 s of synthesized Chinese inquiry streamed into the deployed app at 1× real time (711 frames × 20 ms), against **assemblyai-voice-agent-six.vercel.app**.

0.58 s session.ready after binding the agent	336 ms time to first audio on the greeting turn	2 / 2 tool.call → tool.result round trips	4,488 characters of English quotation produced
6 STT segments from one spoken paragraph	1,524 TTS audio chunks streamed back	53.6 s total session duration	176 unit tests passing

Who pays, and why voice is the right interface

The user

A salesperson at a Chinese phone-accessory factory. They live in WhatsApp voice notes and phone calls all day — typing is the bottleneck, not thinking.

The economics

Quote turnaround drops from tens of minutes to under a minute, and the three money-losing errors (volumetric freight, duty, IP chain) become structural rather than a matter of the salesperson's memory.

Why not a form

Inquiries arrive unstructured and spoken. Voice lets the buyer's own words reach the pricing engine unedited — which is exactly what the system prompt enforces.

Extension path

The same agent definition can be repointed at any rules-and-catalog business: freight forwarding, industrial parts, custom packaging. The voice layer stays; only the tool implementations change.

Where the model is allowed to act

Understanding, tool selection and phrasing. Never arithmetic. That boundary is what makes a voice agent safe to put in front of a customer.

What we think is actually new here

1 · The model is never the source of a number

Most voice-agent demos let the LLM produce the answer. We deliberately routed every figure through a deterministic engine and used the agent only as an interface — then proved it by diffing the spoken summary against the tool output.

2 · We published the protocol bugs

The `reply.done` / `fc-` prefix trap and the Chinese-numeral parser gap are both documented in the repo with the timeline evidence, so the next builder does not lose a day to them.

3 · Tuned for one long utterance

Default VAD split one inquiry into up to 11 fragments. We raised `min_silence` to 1800 ms and added a scene transcription prompt; in the take used in our video the whole paragraph lands in 6 segments and one turn, with the domain terms spelled correctly.

4 · Honest evidence — including what we could not fix

Every claim here comes from a live deployment, the official session timeline, or a passing test. We also publish the limitation: the hosted model still emits a stray clarifying question alongside the first tool call. It survived four prompt rewrites and a `min_silence` change, so we document it instead of hiding it.